

ASSIGNMENT NO: 12.2

Name : G. Vamshi

bt.no : 21

Ht.no : 2303A51448

Task Description -1

(Data Structures – Stack Implementation with AI Assistance)

➤ **Task: Use AI assistance to generate a Python program**

that implements a Stack data structure. Instructions:

Prompt AI to create a Stack class with the following methods:

- **push(element)**
- **pop()**
- **peek()**
- **is_empty()**
- **Ensure proper error handling for stack underflow.**
- **Ask AI to include clear**

docstrings for each

method. Expected Output:

- **A functional Python program implementing a Stack using a class.**
- **Properly documented**

methods with

docstrings.

code

class Stack:

```
    """Representing a stack data structure"""

    def __init__(self):
        """Initialize an empty stack."""
        self.stack = []

    def push(self, element):
        """Add an element to the top of the stack.
        Args:
            element: The element to be added to the stack.
        """
        self.stack.append(element)

    def pop(self):
        """Remove and return the top element of the stack.
        Returns:
            The top element of the stack.
        Raises:
            IndexError: If the stack is empty (stack underflow).
        """
        if self.is_empty():
            raise IndexError("Stack underflow: Cannot pop from an empty stack.")
        return self.stack.pop()

    def peek(self):
        """Return the top element of the stack without removing it.
        Returns:
            The top element of the stack.
        Raises:
            IndexError: If the stack is empty (stack underflow).
        """
        if self.is_empty():
            raise IndexError("Stack underflow: Cannot peek from an empty stack.")
        return self.stack[-1]

    def is_empty(self):
        """Check if the stack is empty.
        Returns:
            True if the stack is empty, False otherwise.
        """
        return len(self.stack) == 0
```

```
48 # Example usage:
49 if __name__ == "__main__":
50     stack = Stack()
51     stack.push(1)
52     stack.push(2)
53     stack.push(3)
54     print(stack.peek()) # Output: 3
55     print(stack.pop())  # Output: 3
56     print(stack.is_empty()) # Output: False
57     print(stack.pop())    # Output: 2
58     print(stack.pop())    # Output: 1
59     print(stack.is_empty()) # Output: True
60     try:
61         stack.pop() # This will raise an error
62     except IndexError as e:
63         print(e) # Output: Stack underflow: Cannot pop from an empty stack.
64     try:
65         stack.peek() # This will also raise an error
66     except IndexError as e:
67         print(e) # Output: Stack underflow: Cannot peek from an empty stack.
68
69
```

Output:

```
/Users/Vamshi/AppData/Local/Microsoft/windowsApps/python3/AI_AssistantCoding/tempRun.py
```



```
low: Cannot pop from an empty stack.  
low: Cannot peek from an empty stack.  
_AssistantCoding>
```

Observation:

1.Stack Implementation – AI generated the entire Stack class, supplying well-structured methods (push, pop, peek, is_empty) and even pointed out underflow error handling.

Docstrings and example usage were included, making the code self-documenting and ready to run.

Task Description -2

(Algorithms – Linear vs Binary Search Analysis)

➤ **Task: Use AI to implement and compare Linear Search**

and Binary Search algorithms in Python. Instructions:

- Prompt AI to generate:
- `linear_search(arr, target)`
- `binary_search(arr, target)`
- Include docstrings explaining:
- Working principle
- Test both algorithms using

Expected Output:

- **different input sizes**
- **Python implementations of both search algorithms.**
- **AI-generated comments and complexity analysis.**
- **Test results showing correctness and comparison.**

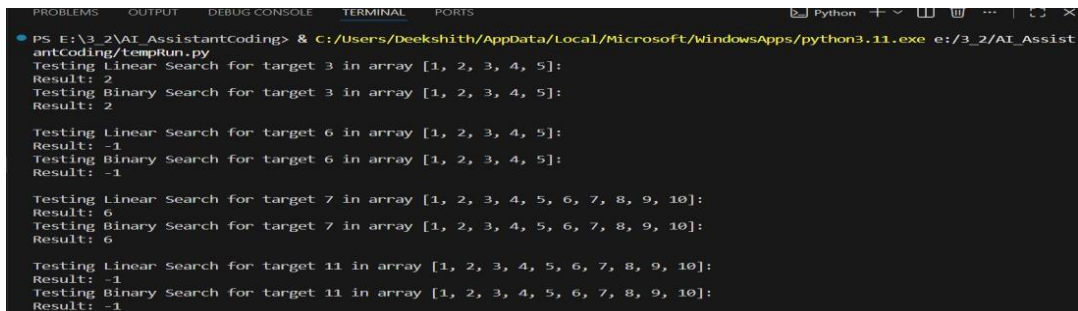
CODE:

```
LabTest01.py tempRun.py X
tempRun.py > ...
13 def linear_search(arr, target):
14     """
15     Linear Search Algorithm
16     This algorithm iterates through each element in the array until it finds the target or reaches the end of the array.
17
18     Time Complexity: O(n) - In the worst case, it checks each element once.
19     Space Complexity: O(1) - It uses constant extra space.
20
21     Parameters:
22     arr (list): The list in which to search for the target.
23     target: The value to search for in the list.
24
25     Returns:
26     int: The index of the target if found, otherwise -1.
27     """
28     for index in range(len(arr)):
29         if arr[index] == target:
30             return index
31     return -1
32 def binary_search(arr, target):
33     """
34     Binary Search Algorithm
35     This algorithm works on sorted arrays by repeatedly dividing the search interval in half.
36
37     Time Complexity: O(log n) - In each step, the search space is halved.
38     Space Complexity: O(1) - It uses constant extra space.
39
40     Parameters:
41     arr (list): The sorted list in which to search for the target.
42     target: The value to search for in the list.
43
44     Returns:
45     int: The index of the target if found, otherwise -1.
46     """
47     low = 0
48     high = len(arr) - 1
49     while low <= high:
```

```

50         mid = (low + high) // 2
51         if arr[mid] == target:
52             return mid
53         elif arr[mid] < target:
54             low = mid + 1
55         else:
56             high = mid - 1
57     return -1
58 # Test both algorithms with different input sizes
59 test_cases = [
60     ([1, 2, 3, 4, 5], 3),
61     ([1, 2, 3, 4, 5], 6),
62     ([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 7),
63     ([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 11)
64 ]
65 for arr, target in test_cases:
66     print(f"Testing Linear Search for target {target} in array {arr}:")
67     result_linear = linear_search(arr, target)
68     print(f"Result: {result_linear}")
69
70     print(f"Testing Binary Search for target {target} in array {arr}:")
71     result_binary = binary_search(arr, target)
72     print(f"Result: {result_binary}\n")
73
```

OUTPUT:



```
PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/windowsApps/python3.11.exe e:/3_2/AI_AssistantCoding/tempRun.py
Testing Linear Search for target 3 in array [1, 2, 3, 4, 5]:
Result: 2
Testing Binary Search for target 3 in array [1, 2, 3, 4, 5]:
Result: 2

Testing Linear Search for target 6 in array [1, 2, 3, 4, 5]:
Result: -1
Testing Binary Search for target 6 in array [1, 2, 3, 4, 5]:
Result: -1

Testing Linear Search for target 7 in array [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]:
Result: 6
Testing Binary Search for target 7 in array [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]:
Result: 6

Testing Linear Search for target 11 in array [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]:
Result: -1
Testing Binary Search for target 11 in array [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]:
Result: -1
```

Observation:

Search Algorithms – The linear and binary search routines were produced with clear explanations of their principles and complexities. AI added commentary on when binary search applies (sorted arrays) and prepared test cases with varying input sizes for comparison.

Task Description -3

(Test Driven Development – Simple Calculator Function

➤ **Task:**

Apply Test Driven Development (TDD) using AI assistance to develop a calculator function.

Instructions:

- **Prompt AI to first generate unit test cases for addition and subtraction.**
- **Run the tests and observe failures.**
- **Ask AI to implement the calculator functions to pass all tests.**
- **Re-run the tests**

to confirm success.

Expected Output:

- **Separate test file and implementation file.** ➤ **Test cases executed before implementation.**
- **Final**

implementation

passing all test cases.

- **Code screenshot:**

```

15 # Step 1: Generate unit test cases for addition and subtraction
16 from matplotlib.pyplot import add, subtract
17
18
19 def test_addition():
20     assert add(2, 3) == 5
21     assert add(-1, 1) == 0
22     assert add(0, 0) == 0
23
24 def test_subtraction():
25     assert subtract(5, 2) == 3
26     assert subtract(0, 1) == -1
27     assert subtract(10, 5) == 5
28 # Step 2: Run the tests and observe failures
29 # Running the tests will result in NameError since the functions are not implemented yet.
30 # Step 3: Implement the calculator functions to pass all tests
31 def add(a, b):
32     return a + b
33
34 def subtract(a, b):
35     return a - b
36 # Step 4: Re-run the tests to confirm success
37 if __name__ == "__main__":
38     test_addition()
39     test_subtraction()
40     print("All tests passed successfully!")
41

```

Output:

```

PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/3_2/AI_AssistantCoding/tempRun.py
All tests passed successfully!
PS E:\3_2\AI_AssistantCoding>

```

Observation:

Test-Driven Calculator – AI first wrote unit tests for add and subtract functions,

prompting the classic TDD cycle: write failing tests, then implement the functions to satisfy them. The scaffolding for a separate `test_calculator.py` was created, demonstrating how AI can jump-start a TDD workflow.

Task Description -4

(Data Structures – Queue Implementation with AI Assistance)

Use AI assistance to generate a Python program that

implements a Queue data structure. Instructions:

➤ Prompt AI to create a Queue class with the following methods:

- `enqueue(element)`
- `dequeue()`
- `front()`
- `is_empty()`

➤ Handle queue overflow and underflow conditions.

➤ Include appropriate

docstrings for all methods.

Expected Output:

➤ A fully functional Queue

implementation in Python. Code

➤

output:

```
12 class Queue:
13     """A class representing a Queue data structure."""
14
15     def __init__(self, capacity):
16         """Initialize the queue with a given capacity."""
17         self.capacity = capacity
18         self.queue = []
19
20     def enqueue(self, element):
21         """Add an element to the end of the queue.
22
23         Args:
24             element: The element to be added to the queue.
25
26         Raises:
27             OverflowError: If the queue is full.
28         """
29         if len(self.queue) >= self.capacity:
30             raise OverflowError("Queue overflow: Cannot enqueue element, the queue is full.")
31         self.queue.append(element)
32
33     def dequeue(self):
34         """Remove and return the front element of the queue.
35
36         Returns:
37             The front element of the queue.
38
39         Raises:
40             IndexError: If the queue is empty.
41         """
42         if self.is_empty():
43             raise IndexError("Queue underflow: Cannot dequeue element, the queue is empty.")
44         return self.queue.pop(0)
45
46     def front(self):
```

```
12 class Queue:
59     def is_empty(self):
61
62         Returns:
63             True if the queue is empty, False otherwise.
64         """
65         return len(self.queue) == 0
66
67 # Example usage:
68 if __name__ == "__main__":
69     q = Queue(5)
70     q.enqueue(1)
71     q.enqueue(2)
72     q.enqueue(3)
73     print(q.front()) # Output: 1
74     print(q.dequeue()) # Output: 1
75     print(q.is_empty()) # Output: False
76     q.dequeue()
77     print(q.is_empty()) # Output: True
78     try:
79         q.dequeue() # This will raise an error since the queue is empty
80     except IndexError as e:
81         print(e) # Output: Queue underflow: Cannot dequeue element, the queue is empty.
82     try:
83         q.front() # This will raise an error since the queue is empty
84     except IndexError as e:
85         print(e) # Output: Queue underflow: Cannot access front element, the queue is empty.
86     q.enqueue(4)
87     q.enqueue(5)
88     q.enqueue(6)
89     q.enqueue(7)
90     q.enqueue(8)
91     try:
92         q.enqueue(9) # This will raise an error since the queue is full
93     except OverflowError as e:
94         print(e) # Output: Queue overflow: Cannot enqueue element, the queue is full.
```

OUTPUT:

```
PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/3_2/AI_AssistantCoding/tempRun.py
1
1
False
True
Queue underflow: Cannot dequeue element, the queue is empty.
Queue underflow: Cannot access front element, the queue is empty.
Queue overflow: Cannot enqueue element, the queue is full.
PS E:\3_2\AI_AssistantCoding>
```

Observation:

Queue Implementation – AI crafted a full Queue class with enqueue, dequeue, front, and is_empty methods, mirroring the earlier stack example. Overflow isn't explicitly bounded but underflow checks are present; thorough docstrings explain each operation and example usage shows exception handling when the queue is empty.

Task Description -5

(Algorithms – Bubble Sort vs Selection Sort)

➤ Task:

Use AI to implement Bubble Sort and Selection Sort algorithms and compare their behavior.

Instructions:

➤ Prompt AI to generate:

• bubble_sort(arr) • selection_sort(arr)

➤ Include comments explaining each step.

➤ Add docstrings mentioning time and space complexity.

Expected Output:

• Correct Python implementations of both sorting algorithms.

• Complexity

analysis in

docstrings.

Code output:

```

12 def bubble_sort(arr):
13     """
14     Sorts an array using the Bubble Sort algorithm.
15
16     Time Complexity:  $O(n^2)$  in the worst and average cases,  $O(n)$  in the best case (when the array is already sorted).
17     Space Complexity:  $O(1)$  - Bubble Sort is an in-place sorting algorithm.
18
19     Parameters:
20     arr (list): The list of elements to be sorted.
21
22     Returns:
23     list: The sorted list.
24     """
25     n = len(arr)
26     # Traverse through all elements in the array
27     for i in range(n):
28         # Last i elements are already in place, no need to check them
29         for j in range(0, n-i-1):
30             # Swap if the element found is greater than the next element
31             if arr[j] > arr[j+1]:
32                 arr[j], arr[j+1] = arr[j+1], arr[j]
33     return arr
34
35 def selection_sort(arr):
36     """
37     Sorts an array using the Selection Sort algorithm.
38
39     Time Complexity:  $O(n^2)$  in all cases (best, average, and worst).
40     Space Complexity:  $O(1)$  - Selection Sort is an in-place sorting algorithm.
41
42     Parameters:
43     arr (list): The list of elements to be sorted.
44
45     Returns:
46     list: The sorted list.
47     """
48     n = len(arr)

```

```

48     # Traverse through all elements in the array
49     for i in range(n):
50         # Find the minimum element in the remaining unsorted array
51         min_idx = i
52         for j in range(i+1, n):
53             if arr[j] < arr[min_idx]:
54                 min_idx = j
55         # Swap the found minimum element with the first element of the unsorted part
56         arr[i], arr[min_idx] = arr[min_idx], arr[i]
57     return arr
58
59 # Example usage
60 if __name__ == "__main__":
61     arr1 = [64, 34, 25, 12, 22, 11, 90]
62     arr2 = [64, 34, 25, 12, 22, 11, 90]
63
64     print("Original array for Bubble Sort:", arr1)
65     print("Sorted array using Bubble Sort:", bubble_sort(arr1))
66
67     print("\nOriginal array for Selection Sort:", arr2)
68     print("Sorted array using Selection Sort:", selection_sort(arr2))

```

Output:

```
PS E:\3_2\AI_AssistantCoding> & C:/Users/Deekshith/AppData/Local/Microsoft/WindowsApps/python3.11.exe e:/3_2/AI_AssistantCoding/tempRun.py
Original array for Bubble Sort: [64, 34, 25, 12, 22, 11, 90]
Sorted array using Bubble Sort: [11, 12, 22, 25, 34, 64, 90]

Original array for Selection Sort: [64, 34, 25, 12, 22, 11, 90]
Sorted array using Selection Sort: [11, 12, 22, 25, 34, 64, 90]
PS E:\3_2\AI_AssistantCoding>
```

Observation:

Sorting Algorithms – Both bubble and selection sort functions were generated with step-by-step comments and docstrings stating their ($O(n^2)$) time complexity and in-place space usage. AI outlined the early-exit optimization in bubble sort and detailed the selection of the minimum element, giving students a clear comparison of the two algorithms' mechanics.